

siox

A hardware description language for the whole circuit

Goal and purpose · October 2026

Abstract. siox (“silicon oxide”) is a hardware description language and compiler. It describes digital circuits with the precise semantics of VHDL and the structure of Rust: entities and processes, nine-value IEEE logic and delta cycles, written with modules, traits, generics and a single strict type system. Designs compile through LLVM to native simulators that run their own tests and write VCD or FST waveforms. The digital language is the first of three phases: analogue and mixed-signal modelling come next, and then turning those models into tool-ready designs. This document explains what siox is for, the principles behind it, and where it is going.

Contents

1	Why another HDL	2
2	Goals	2
3	A first look	3
4	What the language provides	4
5	How it works	4
6	Where siox is going	5
7	What siox will not be	5
8	Summary	5

1 Why another HDL

Hardware engineers today choose between languages that each solve part of the problem.

- **VHDL** has the most careful semantics of the mainstream HDLs. Its nine-value logic, resolved signals and delta-cycle model describe real circuits precisely. Its syntax and its type system were designed in the 1980s, and writing reusable, well-structured code in it is laborious.
- **Verilog and SystemVerilog** are concise and universally supported, but their semantics are loose: four-value logic, implicit widening and truncation, scheduling races between blocks, and a well-known gap between what simulates and what synthesizes. SystemVerilog added a large verification language on top rather than tightening the design language underneath.
- **Embedded generators** such as Chisel and Amaranth use a general-purpose language to build circuits. They gain that language's abstractions, but the circuit is a data structure the host program constructs, so errors, types and tooling belong to the host, not to the hardware.
- **Newer HDLs** such as Spade and Clash bring modern language design to hardware. siox shares their direction and borrows from them where they have already solved a problem (Spade's pipelines are one example).

None of them covers more than the digital part of a circuit. A real board mixes digital logic with analogue parts, power, clocks and the physical connections between them, and today each of those is described in a different tool with a different language.

siox aims to be one language for the whole circuit: precise enough to simulate digital logic exactly, extensible to analogue behaviour, and structured enough to compose both into a design that existing tools can build.

2 Goals

2.1 Hardware semantics first

siox is strict where hardware is strict. Every declaration states its type, assignment and connection widths must match exactly, and arithmetic does not widen silently. A signal has one driver unless its type says how parallel drivers combine. The logic type is the full IEEE 1076-2019 nine-value `std_ulogic` set (U X 0 1 Z W L H -), and uninitialized and conflicting values propagate through vectors and arithmetic as the standard defines. Behaviour is checked against an independent reference simulator, not against siox's own output.

2.2 Two audiences, one language

siox is written to be familiar to two groups at once.

- From **Rust** it takes modules and `use` imports, `pub` visibility, structs and enums, traits with generic implementations, `impl` blocks, expression syntax, newtypes, and a compiler whose diagnostics and lint levels work like rustc's.
- From **VHDL** it takes the hardware model: entities with directional ports, concurrent processes with sequential bodies, resolved signals, labels on processes and generate blocks, and attributes read with the tick (`clk'event`, `x'length`).

Where the two disagree, siox follows Rust for program structure and VHDL for circuit meaning.

2.3 Three sigils, one job each

The punctuation of the language has fixed meanings:

Sigil	Meaning	Example
.	values: fields and method calls	<code>pkt.data</code> , <code>clk.rising()</code>
::	types and modules: paths, variants, associated items	<code>std::logic::Bit</code> , <code>State::Idle</code>
'	attributes: properties of a declaration or signal	<code>sig'event</code> , <code>x'old</code> , <code>probe'keep</code>

2.4 The library over the compiler

Behaviour belongs in source code, not in the compiler. The logic values, their truth tables, numeric vectors and their operators, time units, encodings and conversions are declared in siox's standard library as ordinary types and trait implementations. The compiler provides the mechanisms (type checking, operator dispatch, elaboration, code generation) and evaluates the library's definitions; it does not carry a second copy of what and means for `Logic`. A user can define new value types, operators and literal suffixes the same way the standard library does.

2.5 Tooling that behaves like a modern compiler

sioxc is one compiler invocation per file, like `rustc`. It never runs what it builds. A design compiles to a native object; a file with `#[test]` entities compiles to a native test executable that runs its testbenches, filters them by name and writes waveforms. Every diagnostic carries a stable code, warnings are lints with levels set by `#[allow]/#[deny]` or on the command line, and the same compiler library powers an editor language server. Output is deterministic: the same source always produces the same IR and the same simulation.

2.6 Phased, and honest about it

siox grows in phases and never accepts syntax it cannot yet give meaning to. Analogue constructs are reported as errors in the digital compiler, not ignored. Features that are designed but not built live as written proposals until they land.

3 A first look

An eight-bit counter with a testbench:

```
module counter;

entity Counter {
    clk: Bit in,
    rst: Logic in,
    count: unsigned[8] out,
}

impl Counter {
    let value: unsigned[8] = 0;

    update: process {
        if clk.rising() { // runs only on a rising clock edge
            if rst == '1' { value = 0; }
            else { value = value + 1; }
        }
    }

    count = value; // a wire: always equal to `value`
}

#[test]
entity CounterTest {}

impl CounterTest {
    let clk: Bit = '0';
    let rst: Logic = '1';
    let count: unsigned[8];
    let dut: Counter = { .clk = clk, .rst = rst, .count = count };
}
```

```

clock: process {
    clk = not clk after 5ns;      // free-running clock, 10 ns period
}

stimulus: process {
    await 10ns;                  // hold reset for one edge
    rst = '0';
    for i in 0..9 { await clk.rising(); }
    assert!(count == 10, "counter should reach 10");
}
}

```

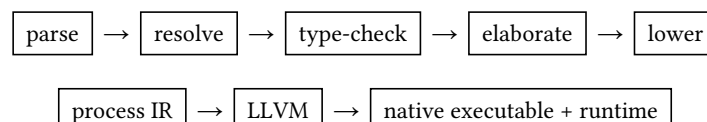
The entity declares the interface; the implementation gives the behaviour. A concurrent assignment (`count = value;`) is a wire; a process is ordered, clocked behaviour. The testbench instantiates the counter like a struct, drives it with timed stimulus and checks it with an assertion. Compiling it with `sioxc --test counter.siox -o tests` and running `./tests -o run.vcd` executes the test and records the waveform.

4 What the language provides

Structure	Entities and implementations, an instance hierarchy, typed ports with directions, struct-shaped buses with directional views, parameterized entities, and structural <code>for/</code> <code>if</code> generation whose labels become named scopes (<code>stages[0].s</code>).
Behaviour	Concurrent assignments and processes, edge detection built from <code>'event</code> and <code>'old</code> , delta-cycle scheduling, simulation time and <code>await</code> , and resolved multi-driver nets.
Types	Nine-value logic, arbitrary-width <code>unsigned/signed</code> , ranged integers, real numbers, characters and strings, structs, enums, arrays, newtypes and transparent aliases.
Abstraction	Traits with generic implementations, one symbol-parameterized operator trait (<code>Operator<"+", In, Out></code>) that also defines new operators, user literal prefixes and suffixes (<code>x"AB"</code> , <code>10ns</code>), and explicit conversions.
Metadata	Declared attributes with defaults, bound from outside the declaration they describe and read back with the tick; <code>#[...]</code> is kept for compiler directives such as <code>#[test]</code> and lint levels.
Verification	<code>#[test]</code> entities, assertions and warnings with source locations, formatted printing, deterministic randomization, file fixtures, and VCD/FST waveforms.
Interoperation	<code>extern "C"</code> functions, an embeddable compiler API, and a language server.

5 How it works

The compiler is one linear pipeline. Each stage hands the next a complete product, and each stage keeps going after an error so a single run reports as much as it can.



Elaboration turns parameterized entities into a concrete instance tree. Lowering produces a digital IR in which combinational drivers and event-controlled updates stay distinct, and every process, hardware or testbench, converges on one control-flow representation. That representation is compiled by LLVM and linked with a small fixed runtime that schedules processes in delta cycles, advances simulation time and

writes waveforms. The simulator is the program the compiler emits, so a design runs at native speed with no interpreter in the loop.

6 Where siox is going

Digital and analogue define components. Design connects and realizes them.

6.1 Phase 1: digital simulation (now)

A strict digital HDL compiling to deterministic native simulation. The baseline is implemented: the language described above, the standard library, native test executables, waveforms, diagnostics and editor support. Ongoing work grows the standard library (synchronizers, memories, FIFOs, numeric families), the scheduler interface, and integration with verification frameworks such as cocotb.

6.2 Phase 2: analogue and mixed signal

Continuous quantities without weakening the digital model: analogue domains, **across/through** relationships, derivatives, equation systems and solvers, and explicit bridges between the digital and analogue worlds with well-defined event ordering. Analogue models get their own IR and solver boundary; the digital IR stays exact and event-driven.

6.3 Phase 3: design and synthesis

Turning component models into designs that existing tools can build: a project and library graph, components written in siox, VHDL or Verilog side by side, constraints and pin assignments, schematic composition, and a vendor-neutral, synthesis-facing representation with adapters for tools such as Vivado, Quartus and Yosys. siox compiles to hardware first; another HDL is only one way of handing that hardware to a tool.

A separate project tool, in the role Cargo plays for Rust, will own packages and builds; the compiler stays a single invocation.

7 What siox will not be

- A package manager or test runner inside the compiler.
- A digital language whose meaning depends on an analogue solver.
- A home for vendor-specific primitives as language keywords.
- A container for foreign HDL syntax inside siox source.
- A language that accepts constructs it cannot yet give meaning to.

8 Summary

siox is a bet that hardware deserves a language with VHDL's precision and Rust's structure, compiled by a toolchain as dependable as a modern software compiler, and that the same language can grow from digital logic to the whole circuit. The digital foundation exists and runs today; the analogue and design phases build on it without changing what the digital language means.